

Battery State of Charge Agentic AI Workshop

Participant Preparation and Hands On Guide

This guide prepares you for a one-hour MATLAB R2026a workshop using an AI coding agent and the experimental embedded-ai-deployment skill. You will work with the MATLAB Day Singapore 2026 battery State-of-Charge project, review evidence produced by the agent, import a PyTorch model, compare a compressed LSTM, and inspect generated C code.

Hardware responsibility Participants do not need an embedded board. The instructor will complete the final deployment demonstration on an STMicroelectronics NUCLEO-F767ZI with an Arm Cortex-M7 processor.

Workshop outcome

- Use an Agent Skill to structure an embedded AI deployment workflow.
- Evaluate a prepared MATLAB-native LSTM State-of-Charge estimator.
- Import and inspect the separate PyTorch MLP model `mlp_soc_model.pt2`.
- Compare the baseline LSTM with the projected and fine-tuned LSTM checkpoint.
- Verify generated behavior on the host before inspecting embedded C code.
- Observe the instructor deploy the compressed LSTM to NUCLEO-F767ZI.

Model path through the workshop

Part	Participant activity	Model used	Result
1	Evaluate the prepared MATLAB model	MATLAB-native LSTM	Baseline SOC accuracy
2	Import and inspect a PyTorch model	<code>mlp_soc_model.pt2</code>	Framework interoperability
3	Compare compression and generate code	Projected and fine-tuned LSTM	Smaller deployment model
Demo	Watch the instructor deploy and profile	Compressed LSTM	Execution on Cortex-M7

Complete before the event

Complete this checklist before arriving. Installation during the workshop is a recovery option and may be blocked by company policy.

- ☐ Bring a Windows laptop and charger.
- ☐ Confirm internet access to GitHub, MathWorks, ChatGPT, and Microsoft Store resources.
- ☐ Confirm that you can sign in to MATLAB and ChatGPT or Codex.
- ☐ Obtain IT approval for local software, PowerShell, Git, and MATLAB Add-Ons.
- ☐ Install MATLAB R2026a and the required products below.
- ☐ Install the ChatGPT desktop app with Codex, or prepare the Codex CLI fallback.
- ☐ Download or clone the workshop repository.
- ☐ Open Aimbdworkshop.prj once before the event.

Required MATLAB products

Type	Required item	Used for
Release	MATLAB R2026a	Agent Skill workflow
Product	Simulink	System integration
Product	Deep Learning Toolbox	LSTM and imported network
Product	Statistics and Machine Learning Toolbox	Project workflow dependency
Product	MATLAB Coder	Host MEX and C code
Product	Simulink Coder	Simulink code generation
Product	Embedded Coder	Embedded C code and reports
Product	Fixed-Point Designer	Embedded numeric workflows
Add-On	Deep Learning Toolbox Model Compression Library	Network projection
Add-On	MATLAB Coder Interface for Deep Learning	Code generation from dlnetwork
Add-On	Deep Learning Toolbox Converter for PyTorch Models	Part 2 model import
Compiler	MATLAB-supported C++ compiler	Desktop MEX verification

Participant hardware tools STM32CubeMX, STM32CubeProgrammer, GNU Arm tools, and the STM32 firmware package are installed only on the instructor machine. Participants do not need these tools.

Install ChatGPT or Codex

Preferred Windows setup

Install the ChatGPT desktop app for Windows before the event. Use the Microsoft Store link or the following Windows Package Manager command if your organization permits it.

```
winget install --id 9PLM9XGG6VKS -s msstore
```

[Official ChatGPT desktop app for Windows](#)

Codex CLI fallback

Use this fallback only when the desktop app is unavailable and your IT policy permits PowerShell downloads and user-local software installation.

```
powershell -ExecutionPolicy Bypass -c "irm https://chatgpt.com/codex/install.ps1 | iex"
```

The command uses the official OpenAI installer. ExecutionPolicy Bypass applies only to the PowerShell process; it does not override company security controls. If installation is blocked, pair with another participant or follow the instructor demonstration.

```
codex
```

At first launch, select Sign in with ChatGPT and complete authentication.

Download the lab files

Clone the repository before the event when possible.

```
git clone https://github.com/KantikaAscendas/Agentic-AI-Embedded-Skill-Workshop.git  
cd Agentic-AI-Embedded-Skill-Workshop
```

[Agentic AI Embedded Skill Workshop repository](#)

If Git is unavailable, use Code then Download ZIP on GitHub, extract the ZIP to a short writable path, and avoid working directly inside the ZIP file.

Install the MATLAB Agentic Toolkit

1. Open the MATLAB Agentic Toolkit repository and download agenticToolkitInstaller.mltbx.
2. Open the downloaded MLTBX file with MATLAB and approve the add-on installation.
3. In MATLAB, run the installer command below.

```
setupAgenticToolkit("install")
```

4. Select only the groups needed for this workshop: MATLAB Core, AI and Statistics, and Code Generation. Include the Simulink Agentic Toolkit because the workshop uses Simulink models.
5. To connect Codex to the MATLAB session that is already open, run the command below in that MATLAB session.

```
shareMATLABSession()
```

[MATLAB Agentic Toolkit installation instructions](#)

Install the embedded AI skill

Open Codex in the workshop repository and send the following prompt. Review and approve the requested installation. Start a new Codex task after installation so the skill becomes available.

```
Install the embedded-ai-deployment skill from https://github.com/matlab/agent-skills-playground/tree/main/skills/embedded-ai-deployment
```

[Agent Skills Playground embedded AI deployment skill](#)

Verify the setup

- Open Aimbdworkshop.prj in MATLAB.
- Run agentic_ai/scripts/workshopPreflight.m.
- Confirm that the final line is READY=true.
- If READY=false, keep the reported missing item visible and ask the instructor before the workshop begins.
- In Codex, ask the verification question below.

What version of MATLAB is running? List the installed toolboxes. Do not modify any files.

Workshop workflow

Time	Activity	Participant evidence
0 to 10	Connect Codex to MATLAB and install or verify the skill	Environment summary
10 to 20	Part 1 baseline LSTM evaluation	SOC prediction and RMSE
20 to 32	Part 2 PyTorch MLP import	Imported network and Simulink integration
32 to 47	Part 3 LSTM compression and host code verification	Size reduction, RMSE delta, MEX result, C report
47 to 57	Instructor NUCLEO-F767ZI deployment	Flash, serial output, memory, execution time
57 to 60	Review and questions	Deployment evidence summary

Start prompt

Use embedded-ai-deployment. Work with this battery State-of-Charge repository using MATLAB R2026a. The project contains prepared train, validation, and test data, a MATLAB-native LSTM, a PyTorch MLP model, a projected and fine-tuned LSTM checkpoint, and Simulink models. The final target is NUCLEO-F767ZI, but participant machines will perform host verification only. Do not modify the original Exercise_1.m, Exercise_2.m, or Exercise_3.m files. Create visible scripts under agentic_ai/generated and results under agentic_ai/results. Work one phase at a time and pause for approval. Begin with environment discovery and a project summary only.

Part 1 prompt

Proceed with the prepared data split and baseline MATLAB-native LSTM evaluation. Do not retrain. Use the existing checkpoint and a representative subset of held-out test data. Report the test count and RMSE, save a prediction plot, then pause.

Part 2 prompt

Proceed with Part 2 using Part_2_AI_import/models/mlp_soc_model.pt2. Use the modern R2026a PyTorch import workflow and rebuild the result as a native dlnetwork suitable for the Cortex-M deployment pattern. Before equivalence testing, propose a small representative test count and wait for approval. Validate against the original PyTorch model, open the native network in Deep Network Designer, use the existing Simulink integration model, report the comparison, then pause. Do not treat this MLP as the final board model.

Part 3 prompt

Proceed with Part 3. The primary goal is minimum flash and model size. Single-precision floating point is acceptable and the final target is NUCLEO-F767ZI. Use the prepared projected and fine-tuned LSTM checkpoint rather than running the long compression sweep or full training. Compare it with the baseline on the same samples, report learnables, estimated parameter bytes, RMSE, and RMSE delta. Generate and validate a desktop MEX implementation before generating library-free C code and opening the code-generation report. Then pause before hardware deployment.

Instructor hardware demonstration

Participants stop after host verification and C-code inspection. The instructor continues with the NUCLEO-F767ZI using the same compressed LSTM validated in Part 3.

Demonstration step	Evidence shown
Select the NUCLEO-F767ZI target	Target configuration and toolchain
Build and flash	Successful build and programmer output
Run SOC inference	Prediction output from the Cortex-M7
Inspect memory	Program flash and RAM usage
Profile execution	Measured on-target inference time

The instructor machine alone requires the STM32 Microcontroller Blockset, STM32CubeMX, STM32CubeProgrammer, GNU Arm toolchain, STM32F7 firmware package, ST-LINK drivers, and the connected NUCLEO-F767ZI.

Troubleshooting

Issue	Action
Company policy blocks installation	Do not bypass policy. Pair with a prepared participant or follow the instructor.
Codex cannot see MATLAB	Run <code>shareMATLABSession()</code> in the MATLAB session you want Codex to use, then retry.
Skill is not available	Start a new Codex task after installation and explicitly say Use embedded-ai-deployment.
Preflight reports a missing item	Install the named MATLAB product or Add-On before continuing.
MEX compiler is missing	Install a MATLAB-supported compiler and configure <code>mex -setup C++</code> .
Build takes too long	Use the prepared checkpoint and instructor-generated artifacts; do not retrain.
OneDrive or antivirus slows builds	Use a short local writable project path for the workshop copy.

Reference links

- [Workshop repository](#)
- [MATLAB Agentic Toolkit](#)
- [Agent Skills Playground](#)
- [Codex CLI guide](#)
- [ChatGPT desktop app for Windows](#)